

Machine Learning Zoomcamp

Module 1 — Introduction to Machine Learning

Study notes with equations and charts

Based on the DataTalksClub course notes, 01-intro

*Condensed and re-organised from
<https://github.com/DataTalksClub/machine-learning-zoomcamp/tree/main/01-intro>. Charts
use synthetic data unless stated otherwise.*

Contents

1	What is Machine Learning?	2
1.1	Features, target, model	2
2	ML vs Rule-Based Systems	2
2.1	The ML recipe	3
2.2	The role reversal	3
3	Supervised Machine Learning	4
3.1	Formal setup	4
3.2	Types of supervised problems	4
4	CRISP-DM: Organising an ML Project	4
5	Model Selection	5
5.1	Hold-out validation	5
5.2	The multiple comparisons problem	6
6	Setting up the Environment	6
7	NumPy	6
7.1	Random arrays	7
7.2	Element-wise, comparison, summarising operations	7
8	Linear Algebra Refresher	8
8.1	Vector–vector (dot / inner product)	8
8.2	Matrix–vector	8
8.3	Matrix–matrix	9
8.4	Identity and inverse	9
9	Pandas	10
9.1	Filtering = boolean masking	10
9.2	Summaries, missing values, grouping	10
10	Summary	11

1 What is Machine Learning?

Motivating problem. On a used-car marketplace a seller must choose a price: too high and nobody buys, too low and money is lost. An expert dealer solves this by having seen many cars: they have *learned patterns from data* (older $\Rightarrow$ cheaper, more kilometres $\Rightarrow$ cheaper, BMW $>$ VW). Machine learning replicates this: an algorithm extracts the same patterns automatically.

1.1 Features, target, model

For each car i we have a **feature vector** $\mathbf{x}_i$ (everything we know about the object) and a **target** y_i (what we want to predict):

$$\mathbf{x}_i = (\text{year, make, mileage, } \dots) \quad y_i = \text{price.}$$

The training data is the set of pairs $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$. Training produces a **model** g , a single artefact that encapsulates the learned patterns. For a new car with unknown price we compute the prediction $\hat{y} = g(\mathbf{x}_{\text{new}})$.

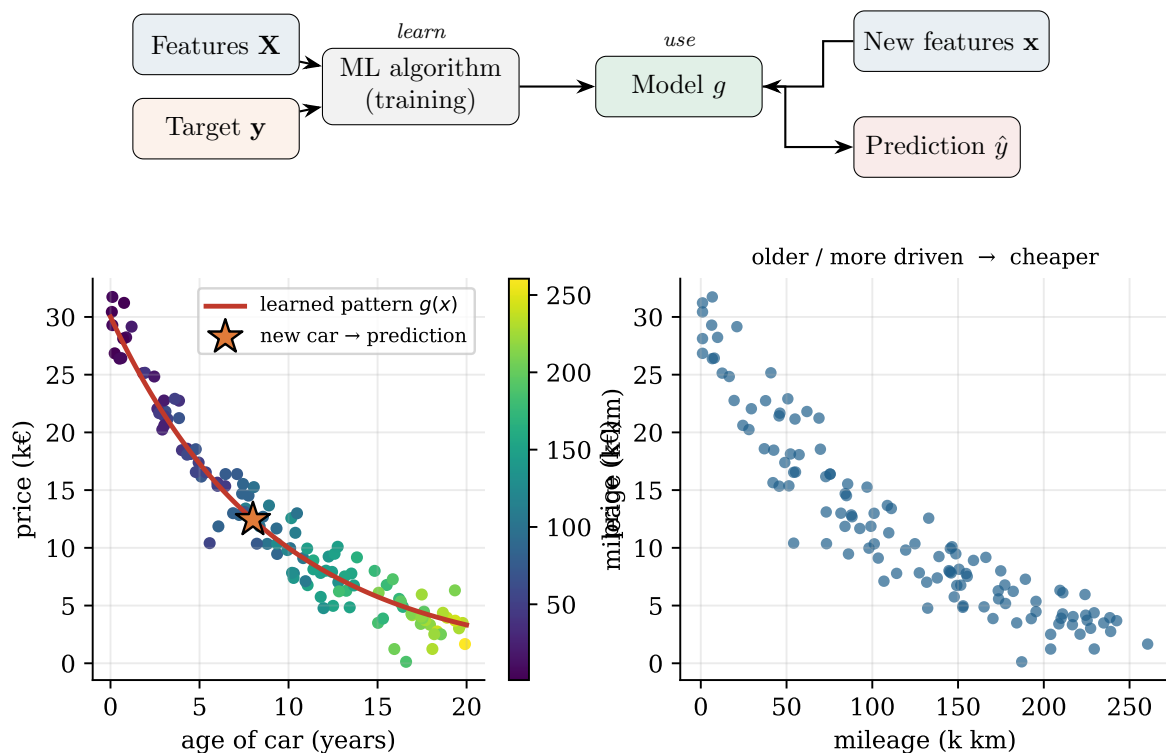


Figure 1: The “pattern” an expert (or a model) learns: price decays with age and mileage. The model is the red curve; a new car (star) gets a price read off the curve. Individual cars scatter around it: predictions are right *on average*, not exactly.

Key idea

ML is the process of **extracting patterns from data**. Data = features + target. Output = a model g . Use = features of a new object $\rightarrow g \rightarrow$ predicted target.

2 ML vs Rule-Based Systems

Example: spam filter. A rule-based approach inspects spam and hard-codes rules:

- if sender = `promotions@online.com` $\Rightarrow$ spam;
- if title contains “tax review” *and* domain = `online.com` $\Rightarrow$ spam;
- if body contains “deposit” $\Rightarrow$ spam ... until a legitimate user writes about a deposit.

Spam keeps evolving, rules multiply, and the code becomes unmaintainable.

2.1 The ML recipe

1. **Get data** — the “Report spam” button provides labelled emails.
2. **Define features** — often the old rules become features! E.g. six binary features:

$$\mathbf{x}_i = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} [\text{len}(\text{title}) > 10] \\ [\text{len}(\text{body}) > 10] \\ [\text{sender} = \text{promotions@online.com}] \\ [\text{sender is hard to read}] \\ [\text{domain} = \text{test.com}] \\ [\text{body contains “deposit”}] \end{pmatrix}$$

where $[\cdot] \in \{0, 1\}$ (true = 1). A spam email might encode as $\mathbf{x} = (1, 1, 0, 0, 1, 1)$ with $y = 1$.

3. **Train and use** — the model outputs a *probability* $g(\mathbf{x}) \in [0, 1]$; a decision needs a **threshold** t :

$$\hat{y} = \begin{cases} 1 \text{ (spam)} & \text{if } g(\mathbf{x}) \geq t \\ 0 \text{ (inbox)} & \text{otherwise} \end{cases} \quad \text{typically } t = 0.5.$$

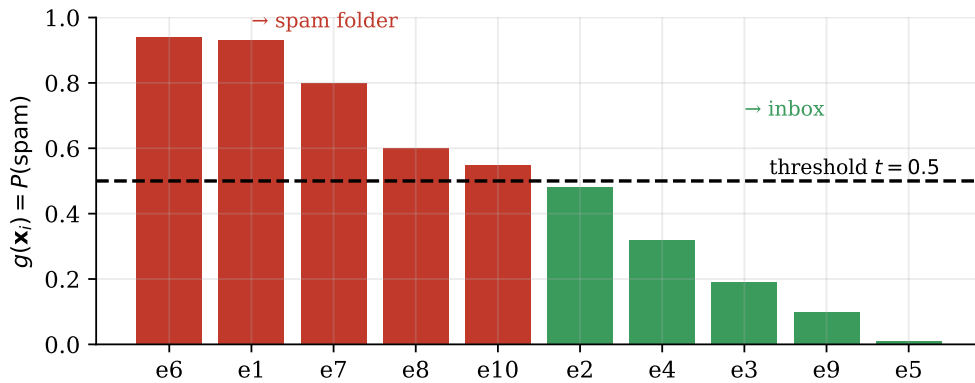
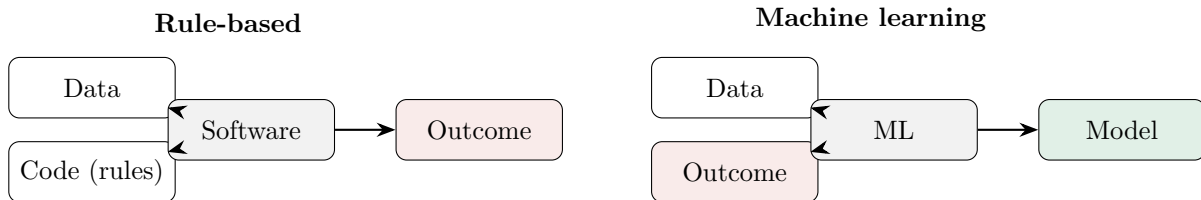


Figure 2: Predicted spam probabilities for ten emails. The threshold converts scores into decisions.

2.2 The role reversal



In classic software the outcome is *hard-coded*; in ML the outcome is an *input*, and the rules are learned.

Tip

Start with a rule-based system. The rules you discover become good features for the later ML system.

3 Supervised Machine Learning

“Supervised” because we act as the teacher: we show examples *with the answer* (label) and the algorithm generalises to new cases.

3.1 Formal setup

With m observations and n features:

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} \in \mathbb{R}^m.$$

Rows = observations (one per email / car), columns = features. The model is a function

$$g : \mathbb{R}^n \rightarrow \mathcal{Y}, \quad g(\mathbf{X}) \approx \mathbf{y}$$

Training = finding g such that its outputs on $\mathbf{X}$ are as close as possible to $\mathbf{y}$. (If the true price is \$11k and we predict \$15k, that may be “close enough”; how we measure closeness comes in later modules.)

3.2 Types of supervised problems

Type	Target space $\mathcal{Y}$	Example
Regression	$\mathbb{R}$ (a number)	car / house price
Binary classification	$\{0, 1\}$, g outputs $P(y = 1) \in [0, 1]$	spam / not spam
Multiclass classification	$\{1, \dots, K\}$, $K > 2$	cat / dog / car
Ranking	scores s_j , then sort	recommendations, search

For ranking, each item j gets a score $s_j = g(\mathbf{x}_j)$ and we show the top- k items: $\text{argsort}(-s)_{1:k}$.

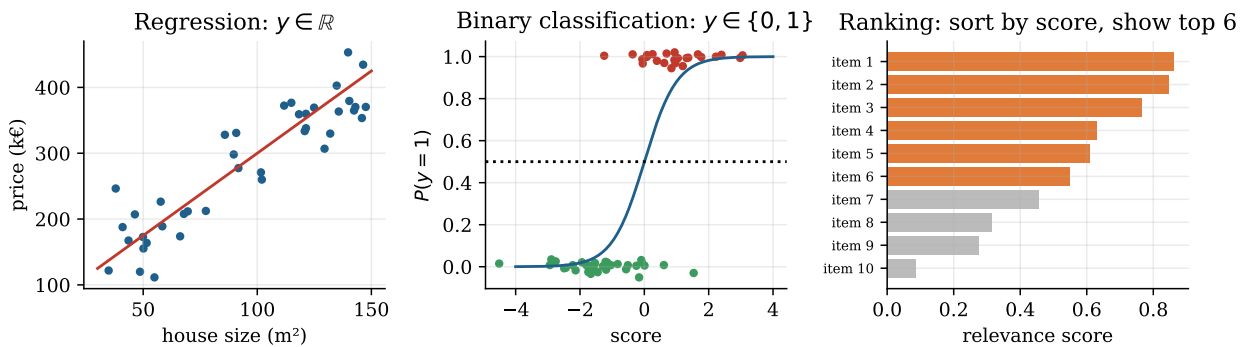
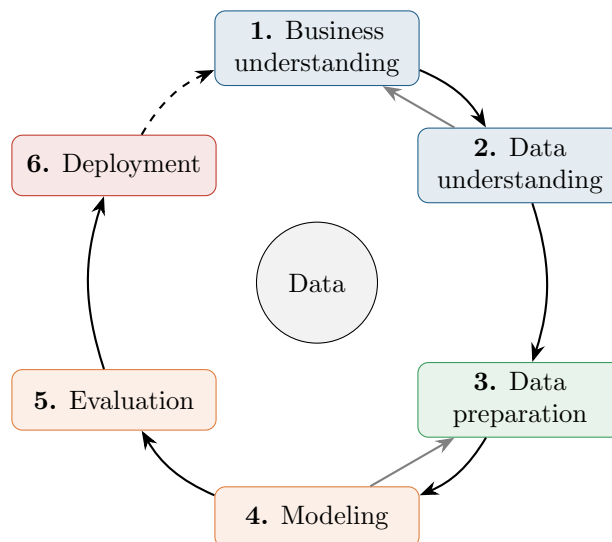


Figure 3: Regression outputs a number, binary classification a probability (thresholded at 0.5), ranking orders items by score.

4 CRISP-DM: Organising an ML Project

CRoss-Industry **S**tandard **P**rocess for **D**ata **M**ining (1990s) — still valid today.



1. **Business understanding.** How big is the problem? *Do we even need ML?* (often a heuristic is enough). Set a *measurable* goal: “reduce spam by 50%”, not “reduce spam”.
2. **Data understanding.** Is the data available, reliable (users may mis-click “spam”), large enough? May send you back to step 1.
3. **Data preparation.** Extract features, clean noise, build pipelines $\Rightarrow$ tabular $(\mathbf{X}, \mathbf{y})$.
4. **Modeling.** Train several models (logistic regression, trees, neural nets, ...), pick the best. Often loops back to step 3.
5. **Evaluation.** Was the business goal reached? 30% instead of 50% — good enough?
6. **Deployment.** Roll out, monitor, maintain. Today evaluation and deployment merge into *online evaluation*: test on e.g. 5% of users first.

Iterate

Start simple $\rightarrow$ go through all steps quickly $\rightarrow$ learn from feedback $\rightarrow$ improve.

5 Model Selection

5.1 Hold-out validation

A model trained in July is used on August data it has never seen. We simulate “the future” by hiding part of the data:



Train g on $(\mathbf{X}, \mathbf{y})$, predict on the validation set and measure **accuracy**:

$$\hat{y}_i = [g(\mathbf{x}_i) \geq 0.5], \quad \text{accuracy} = \frac{1}{m_v} \sum_{i=1}^{m_v} \mathbf{1}[\hat{y}_i = y_i].$$

E.g. 4 correct out of 6 $\Rightarrow$ 66%. Compare candidates (illustrative): LR 66%, tree 60%, forest 67%, NN 80% $\Rightarrow$ pick NN.

5.2 The multiple comparisons problem

Replace the models with *coins*: heads = spam. Each coin's guesses on n validation emails are pure chance, yet with 5 emails a single coin matches all labels with probability $2^{-5} = 1/32$. Trying k coins:

$$P(\text{at least one coin scores 100\%}) = 1 - (1 - 2^{-n})^k.$$

With $n = 5$, $k = 30$ this is already $\approx 61\%$. The “winner” is lucky, not good.

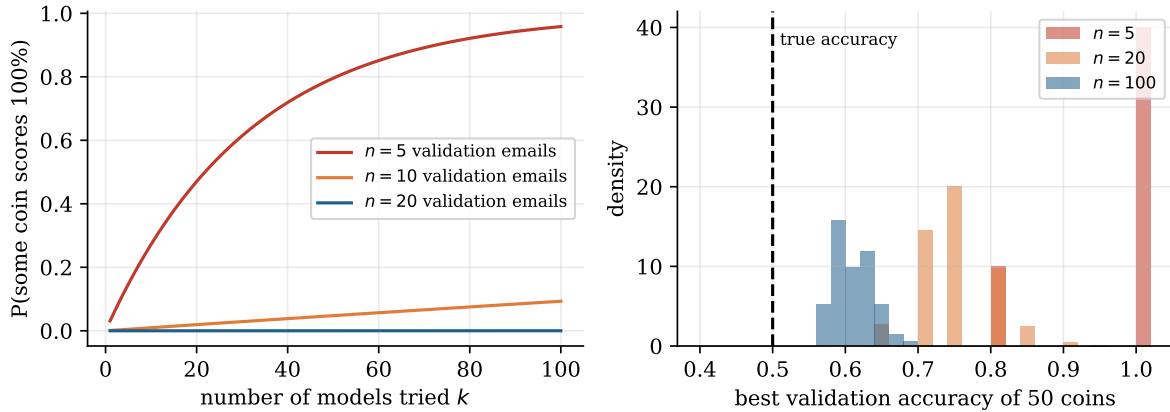


Figure 4: Left: the chance that some useless model looks perfect grows quickly with the number of models tried. Right: best validation accuracy among 50 random coins — far above the true 50%, especially for small validation sets.

Remedy: a second held-out set. After selecting on validation, evaluate once on the test set; if $\text{acc}_{\text{val}} \approx \text{acc}_{\text{test}}$ (e.g. 80% vs 79%) the model is genuinely good.

Recipe

1. Split into train / validation / test (60/20/20 is just an example).
2. Train a model on train.
3. Evaluate on validation.
4. Repeat 2–3 for all candidates.
5. Select the best.
6. Apply it to test; check the score matches validation.

Bonus: after selection, retrain the winner on $\text{train} \cup \text{validation}$ (80% of data), then check on test — validation data is not wasted.

6 Setting up the Environment

Recommended: **GitHub Codespaces** (repo → Code → Codespaces → *Create codespace on main*; works in the browser or VS Code desktop with the Codespaces extension). Needed: Python 3.11, NumPy, Pandas, scikit-learn, Matplotlib, Seaborn, Jupyter.

```
PS1=">" # shorter prompt (optional)
pip install jupyter numpy pandas scikit-learn seaborn
jupyter notebook # port 8888 is forwarded automatically
```

Alternatives: local Ubuntu/WSL, Anaconda, or a cloud VM.

7 NumPy

```
import numpy as np
np.zeros(10); np.ones(10); np.full(10, 2.5) # constant arrays
```

```

a = np.array([1, 2, 3, 5, 7, 12]); a[2] = 10 # from list, 0-indexed
np.arange(3, 10) # [3..9] end exclusive
np.linspace(0, 100, 11) # 11 evenly spaced points: 0,10,...,100
n = np.array([[1,2,3],[4,5,6],[7,8,9]])
n[0, 1]; n[2]; n[:, 1] # element, row 2, column 1 (":" = all rows)

```

7.1 Random arrays

Pseudo-random numbers are reproducible once you fix the seed: `np.random.seed(2)`.

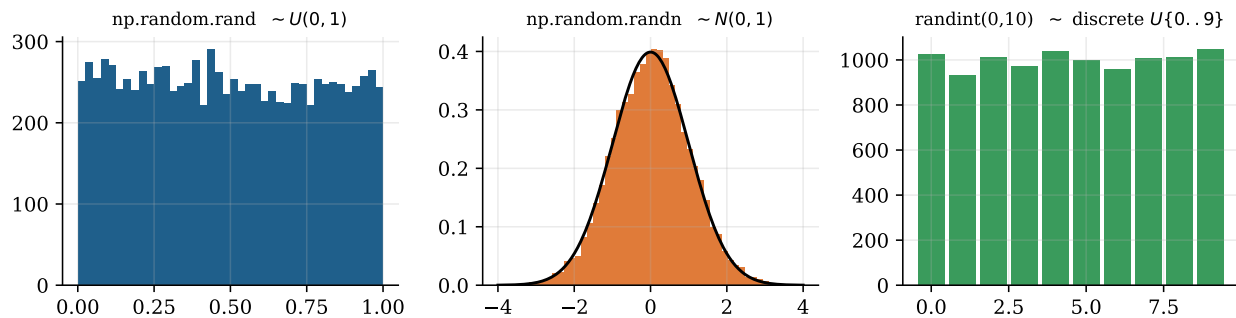


Figure 5: 10 000 draws from each generator. `100*np.random.rand(5,2)` rescales $U(0,1)$ to $U(0,100)$.

7.2 Element-wise, comparison, summarising operations

Element-wise (“vectorised”) operations apply a scalar function f to every entry, or combine two arrays entry by entry:

$$(\mathbf{a} + 1)_i = a_i + 1, \quad (\mathbf{a} \odot \mathbf{b})_i = a_i b_i, \quad \mathbf{b} = \frac{(10 + 2\mathbf{a})^2}{100}.$$

```

a = np.arange(5) # [0 1 2 3 4]
b = (10 + a*2)**2 / 100 # [1. 1.44 1.96 2.56 3.24]
a >= 2 # [F F T T T] boolean mask
a[a > b] # [2 3 4] boolean indexing
a.min(), a.max(), a.sum(), a.mean(), a.std() # 0 4 10 2.0 1.414...

```

Summarising operations reduce an array to one number:

$$\bar{a} = \frac{1}{n} \sum_{i=1}^n a_i = 2, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (a_i - \bar{a})^2} = \sqrt{\frac{4+1+0+1+4}{5}} = \sqrt{2} \approx 1.414.$$

Watch out

NumPy’s `std()` divides by n (population, `ddof=0`); Pandas’ `std()` divides by $n - 1$ (sample, `ddof=1`). Same data, different numbers: for `a` Pandas gives 1.581.

8 Linear Algebra Refresher

Vectors are written as *columns* in maths (NumPy 1-D arrays are just arrays; the convention doesn't change results).

$$\mathbf{u} = \begin{pmatrix} 2 \\ 4 \\ 5 \\ 6 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 2 \end{pmatrix}, \quad 2\mathbf{u} = \begin{pmatrix} 4 \\ 8 \\ 10 \\ 12 \end{pmatrix}, \quad \mathbf{u} + \mathbf{v} = \begin{pmatrix} 3 \\ 4 \\ 5 \\ 8 \end{pmatrix}.$$

8.1 Vector–vector (dot / inner product)

$$\mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i \quad \mathbf{u}^T \mathbf{v} = 2 \cdot 1 + 4 \cdot 0 + 5 \cdot 0 + 6 \cdot 2 = 14.$$

Not to be confused with NumPy's element-wise $\mathbf{u} * \mathbf{v} = (2, 0, 0, 12)$, which is a vector; the dot product sums it into a scalar. The transpose T turns the column $\mathbf{u}$ into a row, so “row $\times$ column”.

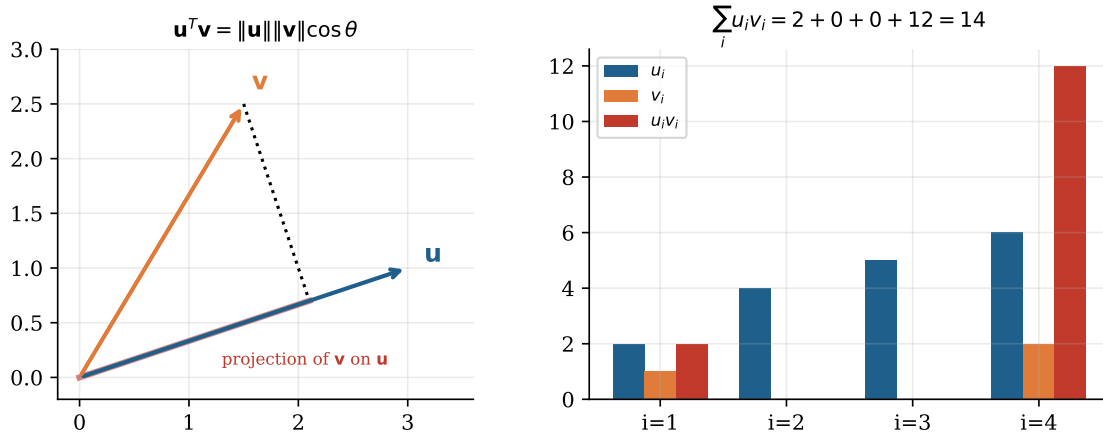


Figure 6: Left: geometric meaning — the dot product measures how much two vectors point in the same direction. Right: the algebraic computation for the example.

```
def vector_vector_multiplication(u, v):
    assert u.shape[0] == v.shape[0]
    result = 0.0
    for i in range(u.shape[0]):
        result += u[i] * v[i] # body of the sum
    return result # == u.dot(v)
```

8.2 Matrix–vector

For $U \in \mathbb{R}^{k \times n}$ and $\mathbf{v} \in \mathbb{R}^n$, each output entry is the dot product of a *row* of U with $\mathbf{v}$:

$$U\mathbf{v} = \begin{pmatrix} U_0^T \mathbf{v} \\ U_1^T \mathbf{v} \\ \vdots \\ U_{k-1}^T \mathbf{v} \end{pmatrix} \in \mathbb{R}^k, \quad (U\mathbf{v})_i = \sum_{j=1}^n U_{ij} v_j.$$

$$\underbrace{\begin{pmatrix} 2 & 4 & 5 & 6 \\ 1 & 2 & 1 & 2 \\ 3 & 1 & 2 & 1 \end{pmatrix}}_{3 \times 4} \underbrace{\begin{pmatrix} 1 \\ 0.5 \\ 2 \\ 1 \end{pmatrix}}_{4 \times 1} = \begin{pmatrix} 2+2+10+6 \\ 1+1+2+2 \\ 3+0.5+4+1 \end{pmatrix} = \begin{pmatrix} 20 \\ 6 \\ 8.5 \end{pmatrix}.$$

Shape rule: $(k \times n)(n \times 1) = (k \times 1)$ — inner dimensions must match (`U.shape[1] == v.shape[0]`).

8.3 Matrix–matrix

Split V into columns; each column of the result is a matrix–vector product:

$$UV = [UV_{:,0} \mid UV_{:,1} \mid \cdots], \quad (UV)_{ij} = \sum_{l=1}^n U_{il}V_{lj}, \quad (k \times n)(n \times p) = (k \times p).$$

$$\left(\underbrace{\begin{pmatrix} 2 & 4 & 5 & 6 \\ 1 & 2 & 1 & 2 \\ 3 & 1 & 2 & 1 \end{pmatrix}}_{\text{row 0 of } U} \right) \left(\underbrace{\begin{pmatrix} 1 & 1 & 2 \\ 0 & 0.5 & 1 \\ 0 & 2 & 1 \\ 2 & 1 & 0 \end{pmatrix}}_{\text{column 0 of } V} \right) = \begin{pmatrix} 14 & 20 & 13 \\ 5 & 6 & 5 \\ 5 & 8.5 & 9 \end{pmatrix}$$

$2 \cdot 1 + 4 \cdot 0 + 5 \cdot 0 + 6 \cdot 2 = 14$

So matrix–matrix $\rightarrow$ many matrix–vector $\rightarrow$ many vector–vector products. In NumPy all three are `.dot()` (or `@`).

8.4 Identity and inverse

The identity I_n has ones on the diagonal, zeros elsewhere (`np.eye(n)`); it plays the role of the number 1:

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad UI = IU = U.$$

For a *square* matrix A , the inverse A^{-1} (if it exists) satisfies $A^{-1}A = AA^{-1} = I$. With the first three rows of V :

$$V_s = \begin{pmatrix} 1 & 1 & 2 \\ 0 & 0.5 & 1 \\ 0 & 2 & 1 \end{pmatrix}, \quad V_s^{-1} = \begin{pmatrix} 1 & -2 & 0 \\ 0 & -\frac{2}{3} & \frac{2}{3} \\ 0 & \frac{4}{3} & -\frac{1}{3} \end{pmatrix}, \quad V_s^{-1}V_s = I_3.$$

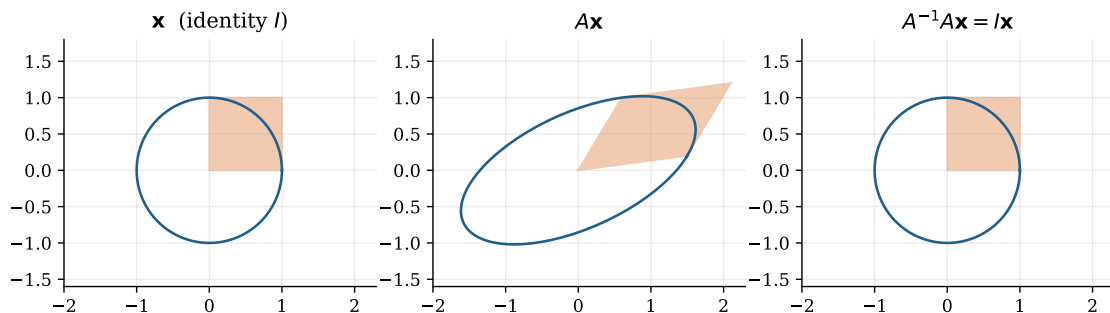


Figure 7: A matrix as a transformation: A stretches and shears the unit circle and square; A^{-1} undoes it exactly.

Why this matters

In Module 2, linear regression is solved with exactly these tools — the *normal equation* $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ uses transposes, matrix products and an inverse.

9 Pandas

A **DataFrame** is a table; each column is a **Series** (a NumPy array + an index + a name). Example data (real, from the course): five cars.

	Make	Model	Year	Engine HP	Cyl.	Transmission	Style	MSRP
0	Nissan	Stanza	1991	138	4	MANUAL	sedan	2000
1	Hyundai	Sonata	2017	NaN	4	AUTOMATIC	Sedan	27150
2	Lotus	Elise	2010	218	4	MANUAL	convertible	54990
3	GMC	Acadia	2017	194	4	AUTOMATIC	4dr SUV	34450
4	Nissan	Frontier	2017	261	6	MANUAL	Pickup	32340

```
df = pd.DataFrame(data, columns=columns) # or from a list of dicts
df.head(n=2)
df.Make; df['Engine_HP'] # brackets needed with spaces
df[['Make', 'Model', 'MSRP']] # several columns -> DataFrame
df['id'] = [1,2,3,4,5]; del df['id'] # add / delete a column
df.loc[1] # by index LABEL df.iloc[[1,2,4]] # by POSITION
df = df.reset_index(drop=True) # returns a new DataFrame
```

9.1 Filtering = boolean masking

A condition produces a boolean Series **m**; indexing keeps rows where $m_i = \text{True}$:

$\text{df}[(\text{df.Make} == \text{'Nissan'}) \ \& \ (\text{df.Year} \geq 2015)] \iff \{i : \text{Make}_i = \text{Nissan} \wedge \text{Year}_i \geq 2015\} = \{4\}.$

```
df['Engine_HP'] * 2 # element-wise; NaN stays NaN
df['Vehicle_Style'] = df['Vehicle_Style'].str.replace('_', '').str.lower()
```

9.2 Summaries, missing values, grouping

```
df.MSRP.mean() # 30186.0
df.MSRP.describe() # count, mean, std, min, 25/50/75%, max
df.describe().round(2) # numeric columns only
df.nunique() # unique values per column (Make: 4)
df.isnull().sum() # missing values per column (Engine HP: 1)
df.groupby('Transmission_Type').MSRP.max() # AUTOMATIC 34450, MANUAL 54990
df.MSRP.values # underlying NumPy array
df.to_dict(orient='records') # back to list of dicts
```

The statistics reported by `describe()`, for MSRP values $x_1, \dots, x_5$:

$$\bar{x} = \frac{1}{5} \sum x_i = 30\,186, \quad s = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2} \approx 18\,985, \quad Q_{50} = \text{median} = 32\,340.$$

`groupby` is SQL's GROUP BY: split rows by key c , apply an aggregate to each group G_c :

$$\text{mean}_c = \frac{1}{|G_c|} \sum_{i \in G_c} \text{MSRP}_i : \quad \text{AUTOMATIC} = 30\,800, \quad \text{MANUAL} = 29\,776.7.$$

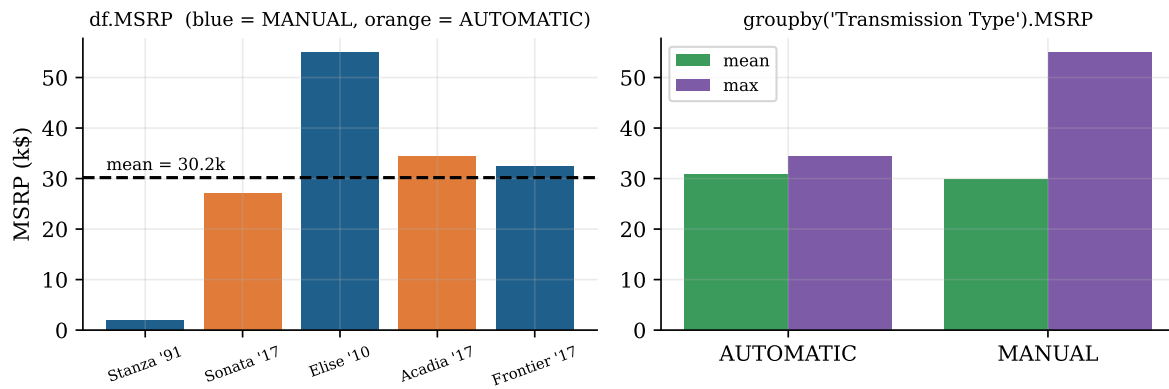


Figure 8: The course's five-car DataFrame: prices with the overall mean, and `groupby` aggregations per transmission type.

10 Summary

What is ML	Learn patterns from features $\mathbf{X}$ and target $\mathbf{y}$; output a model g ; predict $\hat{y} = g(\mathbf{x})$.
Rules vs ML	Rules hard-code the outcome; ML takes the outcome as input and learns the rules.
Supervised ML	Find g with $g(\mathbf{X}) \approx \mathbf{y}$. Regression ($\mathbb{R}$), classification (binary / multi-class), ranking.
CRISP-DM	Business $\rightarrow$ data understanding $\rightarrow$ preparation $\rightarrow$ modeling $\rightarrow$ evaluation $\rightarrow$ deployment; iterate.
Model selection	Train / val / test split; select on val, confirm on test (multiple comparisons problem).
NumPy	Arrays, random generation, element-wise, boolean masks, reductions.
Linear algebra	$\mathbf{u}^T \mathbf{v}$, $U \mathbf{v}$, UV , I , A^{-1} — building blocks of linear regression.
Pandas	DataFrame/Series, <code>loc/iloc</code> , filtering, <code>.str</code> , <code>describe</code> , <code>isnull</code> , <code>groupby</code> .

Next: Module 2 — a real project predicting car prices with linear regression.